TinyJ Assignment 1*

1 About TinyJ

The TinyJ language is an extremely small subset of Java. Every valid TinyJ program is a valid Java program, and has the same semantics whether it is regarded as a TinyJ or a Java program. A piece of source code is a valid TinyJ program if and only if it is both a syntactically valid TinyJ program and a valid Java 8 program.

There are three TinyJ assignments in a sequence. After completing all three assignments you will have a program that can compile any TinyJ program into a simple virtual machine code, and then execute the virtual machine code it has generated. (Execution should produce the same run-time behavior as you would get if you compiled the same TinyJ program using javac into a .class file and then executed that .class file using a Java VM.)

This is the first assignment and the focus is *syntax analysis* of TinyJ programs. At the completion of this assignment, your TinyJ compiler should be able to:

- determine if the sequence of tokens of its input file can be recognized by TinyJ EBNF, and
- output a parse tree, in a "sideways" representation, of the sequence of tokens of its input file.

Please note, a syntactically valid TinyJ program may still contain errors like "undeclared variable" or "array index out of range".

2 TinyJ ENBF

The syntax of TinyJ is given by the EBNF specification that is shown in Figure 1. Reserved words of TinyJ are shown in boldface in this EBNF specification. Some names used by Java library packages, classes, and their methods (e.g., **java**, **Scanner**, and **nextInt**) are reserved words of TinyJ, as is **main**. Otherwise, IDENTIFIER here means any Java identifier consisting of ASCII characters.

3 A Sideways Representation of a Parse Tree

For easy printing of the parse tree, we adopt the sideways representation, where sibling nodes always have the same indentation, but each non-root node is further indented than its parent; the indentation of a node is proportional to the depth of that node in the tree. Figure 2 shows a mapping of the "ordinary" representation to the "sideways" representation of a tree.

4 How to Install TinyJ Assignment 1 on *mars*

You should complete the setup on ***mars*** and test all the assignments on ***mars***. You may choose to do the assignments on your PC/Mac only for your convenience (but success is not guaranteed). For the latter, additional information can be found in "PC-Mac-installtion.pdf".

To install assignment 1, do the following steps:

1. Login to ***mars*** (https://www.cs.qc.cuny.edu/mars/).
2. Create and change to the TinyJ directory (optional, but recommended): `mkdir TinyJ; cd TinyJ`
3. Enter the following on ***mars***: `/home/faculty/ctsai/TJ1setup` (The 1 in TJ1setup is the digit 1, not the letter l.)

*Courtesy of Prof. Tatyung Kong

```
<program>              ::=  [<importStmt>] class IDENTIFIER '{' {<dataFieldDecl>}
                              <mainDecl> {<methodDecl>} '}'
<importStmt>           ::=  import java . util . Scanner ;
<dataFieldDecl>        ::=  static <varDecl>
<varDecl>              ::=  int <singleVarDecl> { ,  <singleVarDecl>} ;
                        |   Scanner IDENTIFIER = new Scanner '('  System . in  ')' ;
<singleVarDecl>        ::=  IDENTIFIER  { '[' ']' } [ = <expr3> ]
<mainDecl>             ::=  public static void main '(' String IDENTIFIER '[' ']' ')'
                              <compoundStmt>
<methodDecl>           ::=  static  ( void | int {'[' ']'})  IDENTIFIER
                              '(' <parameterDeclList>  ')'  <compoundStmt>
<parameterDeclList>    ::=  [<parameterDecl> { ,  <parameterDecl> }]
<parameterDecl>        ::=  int  IDENTIFIER {'['  ']'}
<compoundStmt>         ::=  '{' { <statement> } '}'
<statement>            ::=  ; | return [<expr3>] ; | <varDecl> | <assignmentOrInvoc>
                        |   <compoundStmt> | <ifStmt> | <whileStmt> | <outputStmt>
<assignmentOrInvoc>    ::=  IDENTIFIER ( { '['<expr3>']' } = <expr3> ;  | <argumentList> ; )
<argumentList>         ::=  '('[<expr3>{,<expr3>}]')'
<ifStmt>               ::=  if '(' <expr7> ')'  <statement>  [else <statement>]
<whileStmt>            ::=  while '(' <expr7> ')'  <statement>
<outputStmt>           ::=  System . out . (  print '(' <printArgument> ')' ;
                                            | println '(' [<printArgument>] ')' ;
                                           )
<printArgument>        ::=  CHARSTRING | <expr3>
<expr7>                ::=  <expr6> { '|'  <expr6> }
<expr6>                ::=  <expr5> { &  <expr5> }
<expr5>                ::=  <expr4> { (== | !=) <expr4> }
<expr4>                ::=  <expr3> [ (> | < | >= | <=) <expr3> ]
<expr3>                ::=  <expr2> { (+ | -) <expr2> }
<expr2>                ::=  <expr1> { (* | / | %) <expr1> }
<expr1>                ::=  '(' <expr7> ')' | (+|-|!) <expr1> | UNSIGNEDINT | null
                        |   new int '[' <expr3> ']' { '[' ']' }
                        |   IDENTIFIER ( . nextInt '(' ')' | [<argumentList>]{'[' <expr3> ']'} )
```

Figure 1: TinyJ EBNF production rules. ⟶ indicates non-terminals with incomplete implementation in `Parser.java`.

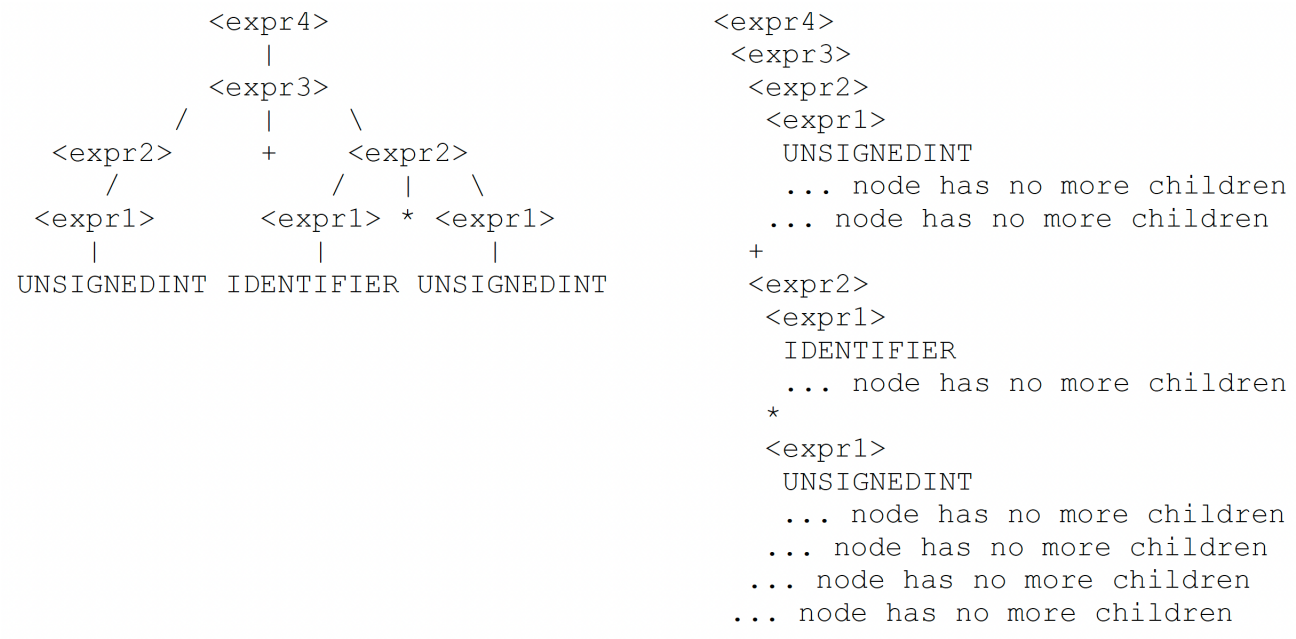

Figure 2: Sideways representation of a parse tree.

4. Wait for the line "TJ1setup done" to appear on the screen.

5. To confirmation correct installation, enter the following command:
   `java -cp TJ1solclasses:.  TJ1asn.TJ CS316ex12.java 12.sol`
   There should not be any error message. `12.sol` should contain a sideways representation of the parse tree for CS316ex12.java.

In both `TJ1asn` and `TJlexer` directories, you should find few `.txt` files. These are the source files of the program with line numbers added. The actual source files without line numbers are in the same directories and have the same names, without `.txt`. The files can be viewed on **mars** using the `less` file viewer. More in-depth explanation of the code can be found in "Assi-1-TJ-Asn-1-Info-code-explanation.pdf".

5 How to Do TinyJ Assignment 1

The file `TJ1asn/Parser.java` is incomplete. It was produced by taking a complete version of that file and replacing parts of the code with comments of the form (or its variations): `/* ????????  */`

You should complete this assignment by replacing every such comment in `TJ1asn/Parser.java` with appropriate code, and recompile the file. On ***mars*** you can use the `nano`, `vim`, or `emacs` editor. To recompile `TJ1asn/Parser.java` do:

```
javac -cp .  TJ1asn/Parser.java
```

6 How to Test Your Parser

In your TinyJ directory, you should find 16 files named `CS316ex`***k***`.java (`***k*** `= 0-15)`. These are all valid TinyJ source files. Execute `TJ1asn.TJ` with each of the 16 files `CS316ex`***k***`.java (`***k*** `= 0-15)` with the command:

```
java -cp .  TJ1asn.TJ CS316ex k.java k.out
```

If your program is correct, each output file ***k***`.out` should be identical to the output file ***k***`.sol` that is produced by running Prof Kong's solution with the same source file as follows:

```
java -cp TJ1solclasses:.  TJ1asn.TJ CS316ex k.java k.sol [on mars and Mac]
java -cp "TJ1solclasses:." TJ1asn.TJ CS316ex k.java k.sol [on PC]
```

To grade your work, I will run `diff -c` to compare the output files by your and Prof. Kong's solutions.

7 How to Submit

To submit:

1. Leave your final version of `Parser.java` on ***mars*** in your `TJ1asn` directory, so it replaces the original version of `Parser.java`, before midnight on the due date.
2. Make a backup copy for Assignment 1: `cp -r TinyJ TinyJ-assign1-bak`
3. Grant the instructor the access permission:
   ```
   chmod 711 $HOME
   cd $HOME
   setfacl -R -m u:ctsai:rwX TinyJ/
   ```
4. Students are allowed to work with other two students, but only one submission per group on Brightspace. Students have to stay in the same group for the entire project (3 assignments).
5. Each submission contains the absolute directory of your working directory and name(s) of student(s). To find such information, do: `pwd` [on ***mars***]
6. Do not touch/open `Parser.java` again after the due date, so not to change the time stamp.